

Nombre del grupo: IRM2526

IRM2526

25/03/26

☺

Miembros del grupo

Miembro1 (ismael.brea@udc.es)

Miembro2 (ruben.gonzalez.ouzounis@udc.es)

Miembro3 (manuel.valina.perez@udc.es)

<https://github.com/GEI-PI-614G010492526/pi2526-irm2526>

Resumen

El proyecto IRM Performance Tracker consiste en el desarrollo de una aplicación web bajo el framework Django que permite analizar el rendimiento de equipos y jugadores de fútbol mediante el uso de Python en el lado del servidor. El sistema integrará datos en tiempo real consultados a APIs externas, para transformar estadísticas brutas en indicadores de valor. Toda la aplicación se desplegará utilizando contenedores de Docker y se gestionará mediante control de versiones en Git, asegurando un entorno de ejecución profesional y colaborativo.

El núcleo de la propuesta reside en el análisis avanzado con la librería Pandas, que se encargará de realizar una limpieza exhaustiva de los datos y calcular métricas críticas como la probabilidad de victoria basada en medias móviles de goles. Además, se procesarán ratios de eficiencia de delanteros (goles por tiro a puerta) para ofrecer una visión profunda del estado de forma de los deportistas. El backend de Django también gestionará una base de datos de usuarios para permitir la personalización de ligas y equipos favoritos.

Para garantizar la calidad del software, el proyecto incluirá testing automatizado de los modelos y de las peticiones a las APIs, evitando fallos en la recuperación de información. La interfaz de usuario estará diseñada para ofrecer un feedback claro, gestionando errores de conexión y visualizando los resultados del análisis de forma intuitiva. Esta práctica final, que representa el 60 % de la nota de la asignatura, se entregará en varias fases marcadas por etiquetas en el repositorio, asegurando un flujo de trabajo iterativo y documentado

Listado exhaustivo de las funcionalidades a implementar

- **F1. Selección de liga:** Permite al usuario seleccionar una liga desde un menú desplegable o buscador.

Entrada: Selección de liga por nombre o identificador.

Salida: Liga seleccionada y carga de equipos asociados.

- **F2. Selección de equipos:** Permite seleccionar dos equipos pertenecientes a la liga elegida, validando que no sean iguales.

Entrada: Selección de dos equipos mediante dropdown o búsqueda.

Salida: Equipos seleccionados y validados para la comparación.

- **F3. Filtrado dinámico de equipos:** Actualiza la lista de equipos disponibles en función de la liga seleccionada.

Entrada: Cambio en la selección de liga.

Salida: Lista de equipos filtrada según la liga.

- **F4. Comparativa de métricas basadas en el rendimiento reciente:** Muestra una comparación entre dos equipos basada en sus últimos 10 partidos.
Entrada: IDs de ambos equipos y datos históricos obtenidos de la API.
Salida: Métricas comparadas como goles a favor, goles en contra y puntos obtenidos.
- **F5. Gráficas comparativas de equipo:** Identifica patrones específicos de rendimiento como resultados de cada jornada.
Entrada: Historial de encuentros recientes y métricas.
Salida: Gráfica de resultados y de estadísticas varias
- **F6. Estimación de probabilidad de resultado:** Genera una predicción basada en las estadísticas calculadas.
Entrada: Métricas agregadas de ambos equipos.
Salida: Probabilidad estimada de victoria, empate o derrota.
- **F7. Consulta de detalle de equipo:** Permite visualizar información detallada de uno de los equipos seleccionados.
Entrada: Selección de equipo.
Salida: Información general y estadísticas del equipo.
- **F8. Consulta de últimos partidos de un equipo:** Permite visualizar el historial reciente de un equipo.
Entrada: ID del equipo.
Salida: Lista de los últimos 10 partidos, ampliables, con resultados.
- **F9. Consulta de estadísticas de jugadores:** Permite visualizar estadísticas individuales de jugadores de un equipo.
Entrada: ID del equipo.
Salida: Lista de jugadores con estadísticas relevantes.
- **F10. Ranking de jugadores por liga:** Muestra rankings de jugadores dentro de una liga.
Entrada: ID de liga.
Salida: Máximos goleadores, máximos asistentes, mayor número de tarjetas rojas y mayor número de tarjetas amarillas.
- **F11. Navegación entre módulos:** Permite moverse entre las distintas secciones de la aplicación.
Entrada: Interacciones del usuario.
Salida: Cambio de vista manteniendo el contexto.

- **F12. Persistencia de estado:** Mantiene la selección de liga y equipos durante la navegación.
Entrada: Acciones del usuario.
Salida: Estado guardado durante la sesión.
- **F13. Obtención de datos desde API-Football:** Recupera datos de ligas, equipos, partidos y jugadores desde la API externa.
Entrada: Peticiones HTTP a la API.
Salida: Datos estructurados en formato JSON.
- **F14. Procesamiento de datos:** Transforma los datos obtenidos en información útil para el sistema.
Entrada: Datos en bruto de la API.
Salida: Datos procesados para visualización y análisis.
- **F15. Gestión de errores:** Maneja errores en peticiones o datos inconsistentes.
Entrada: Fallos en la API o datos inválidos.
Salida: Mensajes de error controlados para el usuario.

Bocetos de las pantallas de la aplicación

- **Mockup 1: Home (Dashborad de Comparativa principal)**
 Vista principal donde se seleccionan dos equipos. Incluye el widget de "Probabilidad de Victoria", métricas comparativas de los últimos 10 partidos y gráficas de rendimiento lineal y de barras para visualizar la eficiencia. Contiene además dos cards Analisis Avanzado y Rendimiento Individual que llevan a dos pestañas nuevas.

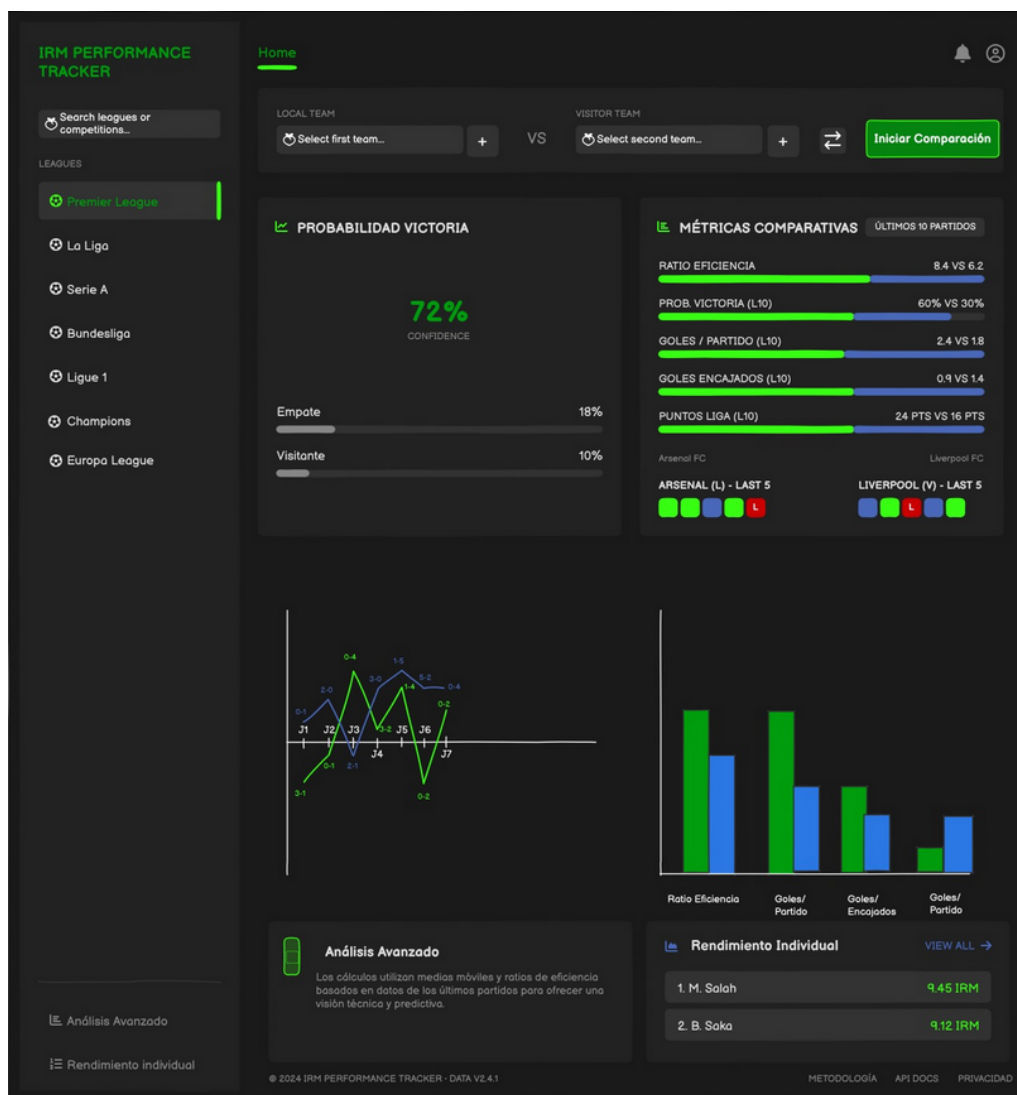


Figura 1: Mockup 1

■ Mockup 2: Rendimiento Individual y Rankings

Panel que muestra los líderes de la liga (máximos goleadores, asistentes, mayor número de tarjetas rojas y mayor número de tarjetas amarillas). Permite una visión global del desempeño de los jugadores en la competición seleccionada.

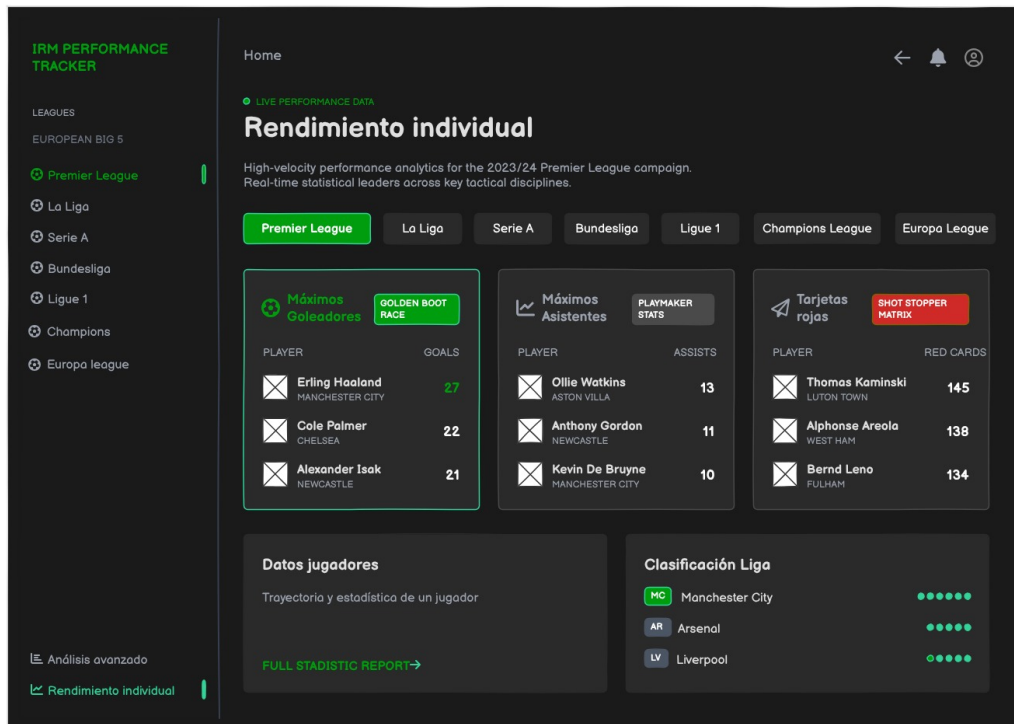


Figura 2: Mockup 2

■ Mockup 3: Detalles del Equipo

Vista profunda de un club específico. Muestra estadísticas básicas de equipo como goles a favor o en contra, los resultados de los últimos 10 partidos, estadísticas promedio (goles a favor/contra) y un desglose de la plantilla (Squad Deep-Dive) con estadísticas individuales de sus jugadores como minutos, goles entre otras. Muestra al final su clasificación actual en liga y tiene abajo un apartado para marcarlo como equipo a comparar que nos lleva a la pantalla principal con el equipo ya seleccionado para comparar (faltaría seleccionar otro).

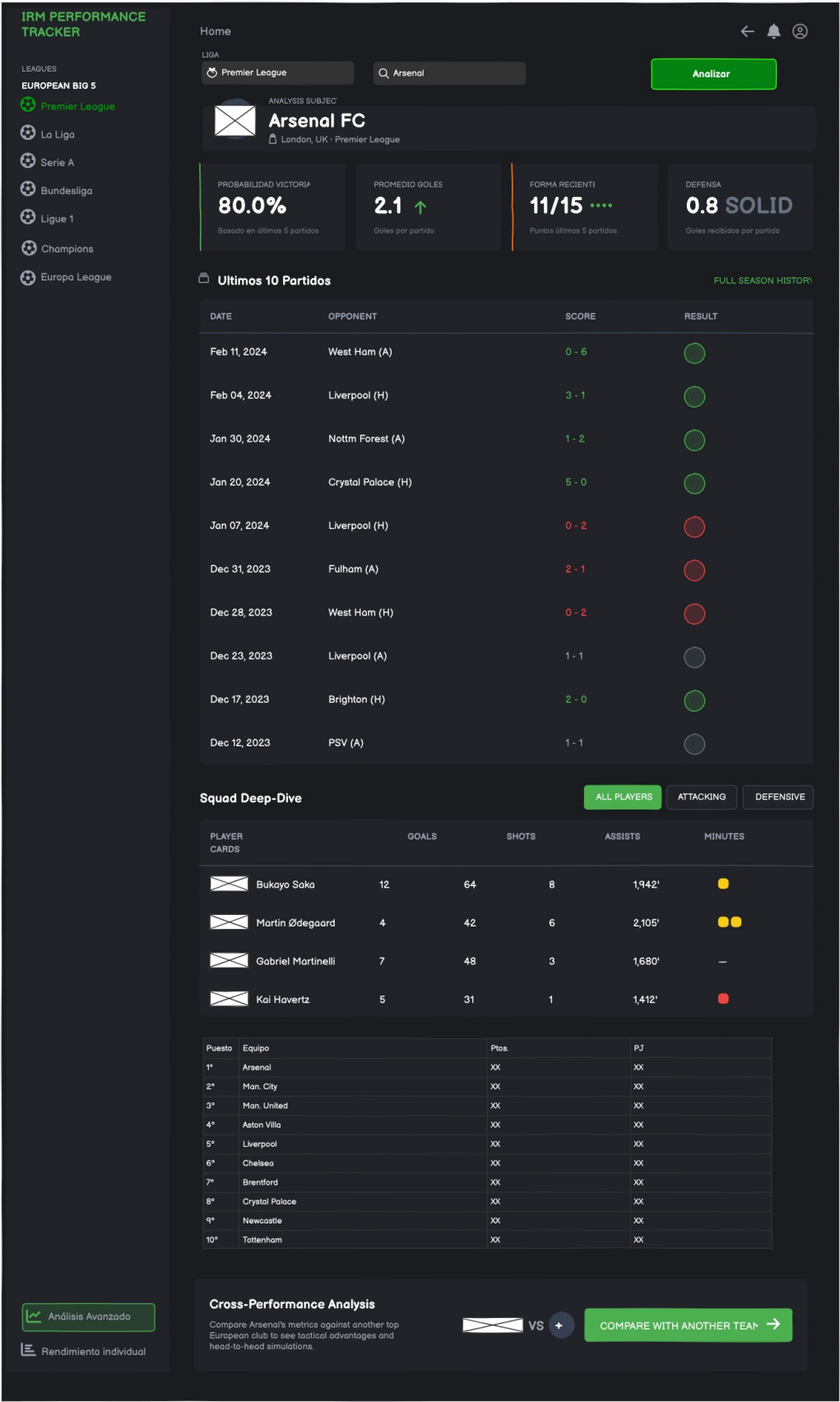


Figura 3: Mockup 3

■ Mockup 4: Datos de un jugador

Interfaz limpia con un buscador central para localizar un futbolista específico y visualizar su trayectoria histórica y estadísticas detalladas por temporada.

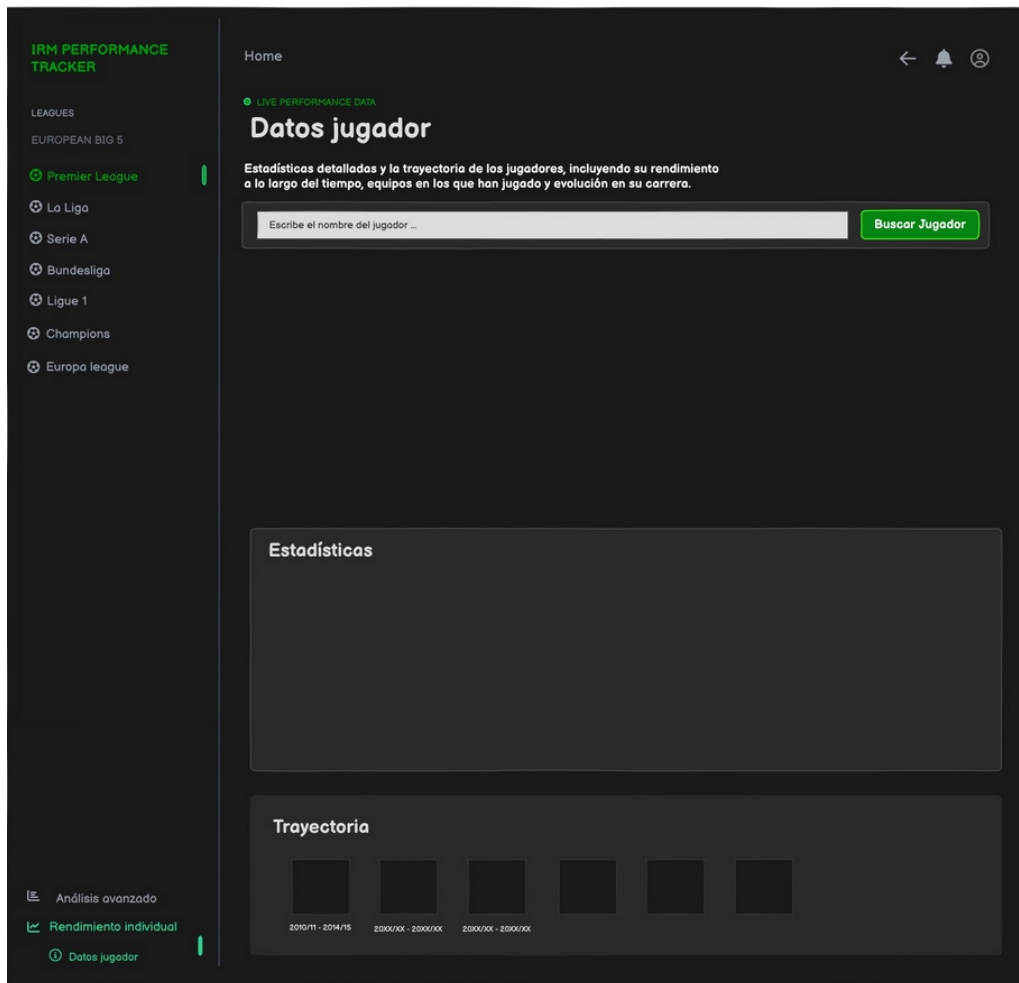


Figura 4: Mockup 4

Flujo de datos de aplicación

El flujo de datos del sistema sigue una arquitectura cliente-servidor basada en Django, donde el backend actúa como intermediario entre la API externa y la interfaz de usuario.

El proceso se desarrolla en las siguientes fases:

1. **Petición del usuario:** El usuario interactúa con la interfaz seleccionando una liga o equipos.
2. **Consulta a la API:** El backend realiza peticiones HTTP a la API-Football para obtener los datos necesarios.
3. **Recepción de datos:** La API devuelve la información en formato JSON.
4. **Procesamiento:** Los datos son procesados mediante Pandas, donde se limpian y se calculan métricas relevantes.

5. **Almacenamiento temporal:** Opcionalmente, los datos pueden almacenarse en caché o base de datos para optimizar futuras consultas.
6. **Envío al frontend:** El backend devuelve los datos procesados a la interfaz de usuario.
7. **Visualización:** El frontend representa los datos mediante tablas, gráficos y métricas comparativas.

APIs utilizadas

- API-Football (vía RapidAPI)

Información a extraer del API

Se utilizarán distintos endpoints de la API-Football para obtener datos necesarios para el análisis de equipos y jugadores:

- **Ligas y equipos**
 - **leagues:** obtención de identificadores de competiciones.
 - **teams:** listado de equipos por liga.
 - **standings:** clasificación actual de la liga.
- **Partidos y rendimiento**
 - **fixtures:** resultados de los últimos partidos.
 - **fixtures/statistics:** estadísticas detalladas por partido.
 - **fixtures/events:** eventos del partido (goles, tarjetas).
 - **fixtures/lineups:** alineaciones y formaciones.
- **Estadísticas de equipos**
 - **teams/statistics:** métricas agregadas (goles, forma, etc.).
- **Jugadores**
 - **players:** búsqueda de jugadores.
 - **players/squads:** plantilla de un equipo.
 - **players/statistics:** estadísticas individuales.
- **Rankings de jugadores**
 - **players/topscorers:** máximos goleadores.
 - **players/topassists:** máximos asistentes.
 - **players/topyellowcards:** jugadores con más tarjetas amarillas.
 - **players/topredcards:** jugadores con más tarjetas rojas.

Uso de Pandas dentro de la práctica

La librería Pandas se utilizará como herramienta principal para el procesamiento y análisis de los datos obtenidos desde la API-Football. Su uso permitirá transformar datos en bruto en métricas avanzadas que aporten valor al sistema.

En primer lugar, se realizará una fase de limpieza y normalización de los datos, eliminando valores nulos, formateando estructuras JSON y adaptando los datos a estructuras tabulares.

Posteriormente, se calcularán distintas métricas de rendimiento, entre las que destacan:

- Media de goles a favor y en contra en los últimos partidos.
- Cálculo de puntos obtenidos (victorias, empates, derrotas).
- Medias móviles de goles para analizar la tendencia de rendimiento.
- Ratio de eficiencia ofensiva (goles por tiro a puerta).
- Indicadores de forma reciente de los equipos.

Además, Pandas permitirá combinar múltiples fuentes de datos (partidos, estadísticas y jugadores) para generar indicadores más complejos, como la estimación de probabilidad de victoria entre dos equipos.

Finalmente, los datos procesados serán estructurados en formatos adecuados para su visualización en la interfaz de usuario, facilitando la generación de gráficas y comparativas.

Funcionalidades a implementar en la primera iteración de la práctica

En la primera iteración (etiqueta v0.1) se implementarán las funcionalidades básicas de conexión y visualización de datos. Concretamente, se desarrollará una vista en Django que permita seleccionar una liga y dos equipos (F1, F2, F3), realice las consultas necesarias a la API-Football (F16) y procese los datos obtenidos mediante Pandas (F17), calculando métricas básicas como goles a favor, goles en contra y puntos en los últimos partidos. El resultado se mostrará en una página con una gráfica comparativa básica entre ambos equipos (F5), sin necesidad de que la interfaz sea completamente usable en este punto.

Uso de librerías externas (opcional)

Como librería *third-party* adicional se utilizará **Plotly.js** para la generación de gráficas interactivas en el cliente, complementando las capacidades de procesamiento de datos realizadas en el backend con Pandas. En concreto, se emplearán gráficas de líneas para representar la evolución del rendimiento de los equipos a lo largo de las jornadas y gráficas de barras para comparar métricas agregadas como goles a favor, goles en contra y puntos obtenidos.

A diferencia de soluciones como Matplotlib, las visualizaciones se generarán en el frontend mediante Plotly.js, permitiendo una mayor interactividad (zoom, hover, filtrado dinámico) y mejor experiencia de usuario. El backend proporcionará los datos procesados en formato JSON, que serán consumidos por el frontend para la construcción dinámica de las gráficas.